

Technical Security

Designing the security for an AI app that isn't built yet — proven with a small demo using fake data. · 10 weeks

Web + Mobile

Biometric MFA

Trains on user + public data

Payments

What the app is. An AI app that learns from a mix of public data and user uploads (text, photos, video, audio) to create personalised experiences. It has user logins, subscriptions/payments, and runs on web and mobile. It doesn't exist yet — so you **design** the security and **prove it with a small proof-of-concept using dummy data**.

Your job. The technical security. **Team 2** handles ethics, privacy and governance. A few topics overlap — see Section 6.

1 Three things that make this app special

It trains a model

New risks a normal app doesn't have: poisoning, model theft, data leakage.

It takes payments

Card data and payment fraud bring PCI rules into scope.

It trains on user data

The model can memorise and leak someone's private information.

2 Ground rules

- **Dummy data only** — never real personal data, card details, or biometrics in your demo.
- **Design first, tools second** — say what must be true before naming a specific technology.
- **Treat every input as hostile** — uploads, API calls, and model inputs are all untrusted.
- **Map to known standards** — OWASP and friends (Section 7), so your choices are defensible.

3 Your priorities

CORE **Must design & demo:** authentication + MFA, data protection, access control, secure communication, logging, risk assessment.

IMPORTANT **Analyse & threat-model:** biometric MFA details, AI/ML model security, payments.

STRETCH **If time allows:** generative-model risks, ML supply chain, adversarial inputs.

Key idea — Biometric MFA

The phone checks the face/fingerprint, **not your server**. So never trust a "biometric OK" message coming from the app — a hacked app can fake it. Instead, the biometric should unlock a secret key **on the device** that signs a challenge from your server. That signature is the real second factor. WebAuthn / passkeys does exactly this.

Key idea — Training on user data

A model can accidentally memorise and repeat private info from its training data — one of your biggest risks here. Also: deleting a user's data is easy in a database but hard once the model has already learned from it. Design an approach and be honest about its limits (don't try to "solve" it).

Key idea — Payments

Don't store card numbers. Route payments through a provider (Stripe-style) with tokenisation, so raw card data never touches your servers. This keeps you out of most PCI scope.

4 What to cover (short version)

Core

- **Auth & identity:** email/password + biometric second factor; safe password reset & account recovery; device enrolment; session management; stop credential stuffing.
- **Data protection:** encrypt at rest and in transit; hash passwords properly; handle uploads safely (scan, limit, strip photo location data).
- **Access control:** users only see their own data; least privilege; controlled admin access.
- **Secure comms:** protect tokens on web and mobile; block common web/API attacks (OWASP).
- **Logging:** log key security events; keep logs tamper-resistant and free of secrets/PII.
- **Risk assessment:** list top risks + controls, and how you'd security-test before launch.

Important & stretch

- **Biometric MFA:** integrate native APIs / WebAuthn; gate a crypto operation, not a boolean; plan recovery without creating a bypass.
- **AI/ML security:** defend against data poisoning; reduce memorisation/leakage; protect the model API; build a deletion mechanism.
- **Payments:** use a PCI-compliant processor; validate payment webhooks; prevent subscription abuse.
- **Stretch:** prompt-injection & unsafe output (if generative), ML supply chain, adversarial inputs.

5 Suggested 10-week plan

Weeks	Focus
1–2	Discovery & threat modelling. Agree overlap points with Team 2.
3–4	Authentication, MFA & access control.
5–6	Data protection & secure communication (web + mobile).
7–8	AI/ML model security + payments.
9	Logging, security-testing plan, build the demo.
10	Finalise demo, diagrams, documentation, present.

6 Working with Team 2

Some topics have two halves: **Team 2 decides the policy**, **Team 1 enforces it**. Agree these early so nothing is missed or done twice.

Topic	Team 2 decides	Team 1 builds
Data classification	Which data is sensitive	Protection for each level
Consent	How consent & withdrawal work	Enforces it; blocks non-consented data from training
Deletion	What "delete my profile" means	The deletion mechanism + its limits
Access control	Who <i>should</i> access data	Enforces roles & least privilege
Audit logging	What must be logged & kept	Tamper-resistant logging
Data provenance	Which sources are allowed	Records origin; validates inputs

7 Standards to use

- **OWASP Top 10 & API Security Top 10** — web & API attacks.
- **OWASP Top 10 for LLM / AI Apps** — AI-specific risks.
- **OWASP MASVS** — mobile app security.
- **WebAuthn / FIDO2** — device-native login done right.
- **STRIDE** — threat modelling. **NIST SP 800-63** — login assurance.

8 Deliverables

- ☐ Security architecture diagram
- ☐ Authentication flow (with biometric MFA)
- ☐ Data flow diagram (including the training pipeline)
- ☐ Threat model / risk assessment
- ☐ Consent-enforcement & deletion mechanism design

☐ Proof-of-concept using dummy data (all Core + one Important/Stretch area)

☐ Documentation of your security decisions & assumptions

Model type (generative vs. classifier) is not yet decided — design generative-model protections as an assumption and flag them. This brief pairs with the Team 2 (Ethics & Governance) brief.